MDA Audio: Uso de gramáticas DSL para el análisis, diseño y generación automática de sintetizadores de audio software

Fernando Real Cisneros
Generación Automática de Código
UNED
Máster Universitario en Investigación en Ingeniería de Software

13 de mayo de 2026

Resumen

Este artículo investiga sobre la viabilidad de utilizar modelos de razonamiento automático para la generación de código fuente mantenible. Para ello, sostiene dos tesis fundamentales: en primer lugar, es necesaria la aplicación de modelos estrictos y restringidos al ámbito específico del software a generar, estructurando tanto la fase de análisis como la de diseño para facilitar su interpretación por parte de la inteligencia artificial. En segundo lugar, debemos alejarnos de la estereotipación de lenguajes de modelado genéricos, como UML, priorizando el uso de Lenguajes de Dominio Específico (DSL) propios del ámbito de aplicación (entendiendo ámbito de aplicación ejemplos como la generación de software, gestión de pedidos, comercio electrónico, etc), independientes de la implementación técnica pero estrechamente ligados al dominio del problema. A partir de estos preceptos, el artículo toma como marco metodológico la Arquitectura Dirigida por Modelos (MDA), y como ámbito ejemplo el desarrollo de software para sintetizadores de audio. Como resultado, se ha diseñado un conjunto de DSL y un Entorno de Desarrollo Integrado (IDE) denominado MDAA (*Model Driven Development Audio*), capaces de abstraer el conocimiento del dominio y transmitir información altamente estructurada a un LLM genérico, permitiendo así la generación automática del código fuente objetivo, manteniendo a su vez la trazabilidad entre los distintos niveles de abstracción.

Palabras clave— MDA, LLM, Generación automática de código, DSL, Síntesis de audio, Ingeniería de Software.

I. Introducción

El presente proyecto de investigación tiene como finalidad principal valorar si la estructuración del desarrollo de sintetizadores de sonido mediante la aplicación de *Model-Driven Architecture* (MDA), adaptando el modelado a las necesidades del dominio de la música electrónica, y orientándolo a la generación automática de código a través de la integración con herramientas de IA, puede aportar mejoras significativas en el desarrollo de este tipo de software. Para lograrlo, las herramientas desarrolladas se centrarán en las dos primeras capas de abstracción que propone MDA: el *Computation Independent Model* (CIM) y el *Platform Independent Model* (PIM).

Con el objetivo de orientar y formalizar estos desarrollos, se han diseñado e implementado una serie de lenguajes de dominio específicos (DSL) al ámbito en cuestión, 2 para el modelado CIM y otros 2 para el modelado PIM. Estos lenguajes permiten capturar la semántica y la arquitectura de los desarrollos en 2 diferentes niveles de abstracción. Como novedad, respecto al uso convencional de la capa CIM por otras herramientas, donde la escritura de los requisitos se representa de manera desestructurada, en este trabajo se propone una estructuración cerrada y estricta para esta capa, pero sin perder el nivel de abstracción que requiere esta capa.

Una de las grandes ventajas de esta estructuración basada en modelos estrictos para las capas CIM / PIM, y su relacionabilidad entre ellas, es tanto la capacidad de generar *prompts* altamente estructurados y fáciles de procesar por inteligencias artificiales como la de poder aplicar técnicas de ingeniería inversa entre los distintos modelos desarrollados. Esta integración brinda la posibilidad de iterar de manera continua y eficiente entre las fases de diseño y análisis de los modelos desarrollados, permitiendo obtener con ello, de forma iterativa, nuevas posibles generaciones del código fuente objetivo.

Para materializar este enfoque, se ha desarrollado una herramienta integral, llamada MDAA, que implementa, a modo de IDE de desarrollo, todo el proceso de análisis y diseño para el dominio ejemplo, cuyo código fuente se encuentra en la carpeta *_CODIGO/* del paquete entregado, y también en el repositorio GitHub[1] del proyecto. Además, aprovechando el acceso gratuito para estudiantes de UNED en Azure, se puede acceder a ella de forma remota a través de enlace web[2] (horario de activación, de 8:00 a 20:00, de lunes a viernes), usando el usuario y contraseña `test@uned.es` / `11111111`. Esta plataforma ofrece un entorno completo para la definición visual de los modelos, facilitando el ciclo de vida del desarrollo y comprobación de las tesis sostenidas por este artículo. Los principales puntos que ofrece la herramienta son:

- **Gestión de modelos CIM y PIM:** Entornos dedicados para la implementación de los modelos de análisis y diseño. Desarrollo gráfico de los modelos PIM, facilitando su implementación mediante la abstracción de los aspectos de bajo nivel postulados por los DSL desarrollados.
- **Lenguajes DSL especializados:** Integración de gramáticas propias que validan la coherencia estructural de las conexiones y flujos de señales que debe tener una aplicación software de estas características.
- **Exportación asistida para IA:** Capacidad de exportar el contexto del proyecto junto con los manuales de los DSL, adaptándolos de cara a generar un prompt estructurado y coherente, orientado a guiar a una IA en la generación, de forma estricta, del código deseado.
- **Validación continua:** Verificación en tiempo real para garantizar que los modelos cumplen con las restricciones del dominio antes de su implementación.

Aparte, en el mismo repositorio de la herramienta, en la carpeta *_CODIGO/dsl*, se puede encontrar información detallada de los modelos DSL desarrollados y su código. Los DSL desarrollados constituyen el núcleo de este proyecto de investigación, intentando buscar con ellos una forma de estructurar desarrollos en este dominio concreto.

[1] `https://github.com/ferquete/mdaa-uned`

[2] `http://mdaa-app.spaincentral.azurecontainer.io/`

En el repositorio entregado, disponemos tanto de un manual completo de la aplicación IDE mencionada, MDAA[3], como de los DSL[4] desarrollados.

Nota importante: Todo documento, enlace de código o información mencionados en este artículo como accesible bajo el repositorio del proyecto[5], se encuentra disponible también, bajo la misma ruta, en el .zip adjunto.

Recomendación: Debido a la especificidad de la materia, se recomienda consultar el manual complementario[6] sobre síntesis de sonido y Pure Data. Este documento, realizado ex profeso para lectores no familiarizados con el dominio abordado en este artículo, ofrece una visión general sobre la síntesis de sonido digital y unas nociones básicas de lo que es *Pure Data*, el lenguaje utilizado en las pruebas.

[3] https://github.com/ferquete/mdaa-uned/blob/main/_DOCUMENTACION/manual_mdaa/latex/main.pdf

[4] https://github.com/ferquete/mdaa-uned/blob/main/_DOCUMENTACION/manual_dsl/latex/main.pdf

[5] https://github.com/ferquete/mdaa-uned

[6] https://github.com/ferquete/mdaa-uned/blob/main/_DOCUMENTACION/material_complementario/latex/main.pdf

II. El Problema

II.A. Descripción del problema

Se puede afirmar sin lugar a dudas que el dominio de problema ejemplo, desarrollo de software orientado a la creación de sintetizadores de sonido, constituye un ámbito de programación altamente especializado que, con frecuencia, se desvía de los paradigmas clásicos de programación (declarativos u orientados a objetos). Históricamente, este campo ha carecido de metodologías formales que estructuren el ciclo de vida del desarrollo, priorizando una aproximación directa a la implementación sobre un proceso sistemático de ingeniería. En esto ha pesado mucho el usuario al que los lenguajes especializados en este campo se dirigen: músicos, ingenieros de sonido, diseñadores de sonido, etc., los cuales no suelen tener conocimientos de programación o ingeniería de software. Salvo en los casos de lenguajes de propósito general, como librerías adaptadas a la síntesis musical en JavaScript o C, u otras enfocadas a la investigación sonora como SuperCollider o Csound, la mayoría de las herramientas disponibles con más éxito comercial y entre los usuarios para el desarrollo de este tipo de software son frameworks de desarrollo visual, como Pure Data o Max/MSP.

El problema fundamental radica en la ausencia de un flujo de trabajo que permita asentar un análisis sólido, el cual deba desembocar en un diseño arquitectónico que sirva como pilar para la generación del código fuente. Esta carencia metodológica impide la trazabilidad y la iteración efectiva entre las distintas fases del desarrollo. Sin un modelo formal intermedio, resulta extremadamente complejo volver atrás para redefinir el análisis o aplicar cambios en el diseño de forma que se reflejen de manera coherente y automática en el producto final. Esta desconexión entre la intención creativa y la implementación técnica limita la escalabilidad de los proyectos y dificulta la reutilización de componentes lógicos entre diferentes desarrollos.

Por otro lado, el surgimiento de LLMs capaces de generar código de alta calidad ha abierto nuevas posibilidades para el desarrollo de software. Sin embargo, el uso de estos modelos de cara a la generación automática de código también presenta grandes desafíos, como la falta de mecanismos formales para realizar *Backward Engineering* entre los modelos definitorios del código y el propio código generado por la propia IA, y cómo se puede mantener la trazabilidad entre ambos. Se hace necesario buscar mecanismos que permitan unir la intención del desarrollador con el código generado por la IA, a través de una representación intermedia que sea fácilmente comprensible por el ser humano y que a la vez sea capaz de ser utilizada por una IA para generar el código esperado, fiel a lo que se pretende en su análisis y diseño.

Auque ya se dispone de multitud de lenguajes de modelado, como UML, y técnicas muy efectivas de generación de código a partir de los mismos, se hacen demasiado generalistas para poder ser utilizados en ámbitos alejados de la ingeniería de software.

II.B. Restricciones introducidas al problema genérico

En este artículo se abordará la generación de código a través de modelos de los mismos, y su viabilidad funcional, y no tanto en la transformación entre los distintos modelos que analizan y diseñan el código, pero todas las herramientas desarrolladas para este trabajo tienen esa finalidad última: que tanto el análisis como el diseño y la implementación del código sean meros artefactos generados automáticamente a partir de unas especificaciones iniciales, en un proceso iterativo de refinamiento de las mismas, con trazabilidad total entre los distintos modelos generados.

II.C. Estado del arte y trabajos relacionados

Si bien es cierto que es posible utilizar lenguajes de propósito general (como C++ o Rust) para el desarrollo de sintetizadores de audio digital, lo que permitiría teóricamente aplicar en su desarrollo técnicas como las postuladas por *Model-Driven Architecture* (MDA) y ciclos de vida de ingeniería de software tradicionales, este no es, como ya mencionamos antes, un enfoque realista en la práctica.

Para los usuarios promedio que desarrollan y utilizan este tipo de software, el uso de lenguajes de bajo nivel o marcos de trabajo genéricos supone una barrera de entrada técnica insalvable

que asfixia la creatividad. Por ello, como ya introducimos previamente, la industria ha convergido hacia herramientas de programación visual, que aunque facilitan la implementación inmediata, carecen habitualmente de las capas de abstracción necesarias para realizar un análisis y diseño formal previo e independiente de la implementación final.

Por todo ello, se hace interesante este dominio de ejemplo, en contraposición a otros más generalistas y bien conocidos por los ingenieros de software, como pudieran ser el desarrollo de aplicaciones de gestión o el de plataformas e-commerce. No parece ser el típico campo de desarrollo en el que se suela aplicar MDA; sin embargo, apoyándose en DSL especificos y el uso de modelo de razonamiento automático, se presta especialmente a ello, abriendo un camino para el desarrollo de este tipo de software que no explorado antes.

III. Cómo evaluar la solución

En esta fase del proyecto, en la cual se persigue generar código fuente automáticamente, validando o no la hipótesis planteada, el enfoque se centrará en comparar ejemplos desarrollados por la herramienta MDAA[7].

III.A. Metodología de Evaluación

Se partirá de 4 ejemplos desarrollados en la herramienta MDAA, disponibles en el repositorio de GitHub[8]. En ese mismo directorio se encontrará un README.md y una serie de scripts para que puedan ser validados contra los DSL desarrollados, e incluidos en la BD local si se decide cargar la aplicación MDAA localmente.

Como se comenta en el manual de la aplicación MDAA, por cada uno de estos 4 proyectos, se podrá generar un paquete .zip con el formato adecuado para el procesado del mismo por una IA, de cara a la obtención del código fuente objetivo. Dentro de este .zip se encontrará un fichero README.md, el cual tendrá un apartado llamado *Suggested Prompt*, que será el prompt a proporcionar para que la IA trabaje, además de adjuntarle todo el fichero .zip.

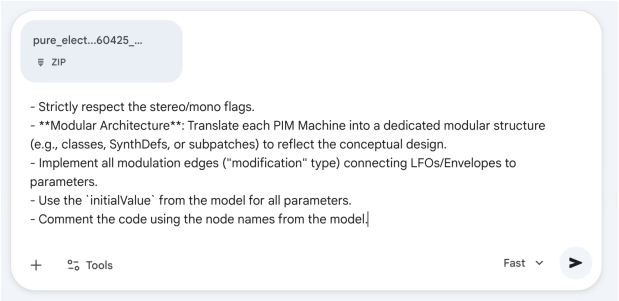

Figura 1: Ejemplo de puesta a punto de prompt

Para esta evaluación de resultados, se comparará en base a generación de código en formato Pure Data, ya que es uno de los más complejos de generar por las IA generalistas. En el manual complementario sobre sonido digital[9], se puede encontrar información sobre este formato y el porqué de su complejidad para su generación automática por parte de modelos LLM.

III.B. Criterios de Análisis

Para la evaluación de la herramienta, en su estado actual, se ha optado por realizar una evaluación cualitativa preliminar y de carácter puramente heurístico. Dado un caso de prueba, se deberían valorar los siguientes aspectos de cara a decidir si la generación es óptima respecto a un desarrollo manual (sin MDAA y sin intervención de IA) equivalente:

- **Replicabilidad:** Evaluar si múltiples intentos de pasar la misma información al mismo modelo LLM y/o a otros dan resultados parejos. Para ello, si de 10 intentos de generación correctos, en varios chats distintos con memorias separadas, se consigue un sonido casi exacto, se considerará que su replicabilidad es Excelente; si varía pero es consistente con la descripción del proyecto, Muy Buena; si la variación es notablemente visible, pero el resultado es funcional, Buena; si no es consistente con la descripción del proyecto, Mala.

- **Grado de Calidad:** Valorar si el resultado es óptimo para el propósito buscado. Aunque este criterio es el que más componente subjetiva posee, permite medir la alineación del código con la intención del diseñador. Podrá tener los valores Correcto o Incorrecto.

- **Número de Intentos:** Determinar cuántos intentos son necesarios para obtener un resultado válido respecto a los criterios de análisis y diseño de ese proyecto concreto. Se darán por buenos aquellos intentos que solo deban ser modificados ligeramente, por ejemplo, por falta de una conexión de interacción con el usuario (como podría ser la falta de una arista en un botón pulsador hacia un objeto de Pure Data).

[7] Manual MDAA en GitHub
[8] Casos de prueba en GitHub
[9] Manual complementario en GitHub

III.C. Descripción de alto nivel de la solución aportada

Para la implementación del trabajo realizado, de cara a poder desarrollar las fases CIM y PIM de MDA en el ámbito seleccionado como ejemplo (el desarrollo de sintetizadores de audio software), se han implementado 4 DSLs en lenguaje Langium, y un IDE de desarrollo completo, ya mencionado en el apartado de introducción, llamado MDAA (Model-Driven Development Audio), que implementa estos 4 DSLs, y sirve de soporte para su generación.

IV. Descripción detallada de la solución propuesta

En el repositorio del proyecto, se encuentra ampliamente documentada la definición de estos 4 DSLs desarrollados (véase Manual DSL[10]), por lo que el presente apartado se limitará a exponer las ideas clave detrás del diseño de los mismos.

Lenguajes de Análisis (CIM)

- `mdaa-cim-machine`: Permite la definición conceptual de máquinas de audio, estableciendo su identidad, puertos de comunicación y configuraciones base. Una máquina, concepto ampliamente expuesto en la documentación de los DSL, se puede considerar como un conjunto de elementos que interactuan entre si, pero que tienen una finalidad común, que es la finalidad de la máquina en su conjunto.

- `mdaa-cim-relations-machines`: Define las conexiones conceptuales y el flujo de señales, a nivel abstracto, entre las distintas máquinas definidas en el análisis.

Lenguajes de Diseño (PIM)

- `mdaa-pim-machine`: Facilita la definición detallada de la arquitectura interna de las máquinas, especificando generadores de audio, modificadores de los parámetros o señales de los generadores, y efectos de procesado de señales de audio.

- `mdaa-pim-relations-machines`: Se encarga de la orquestación de las interconexiones técnicas y vínculos detallados entre los componentes internos de las máquinas.

La idea clave de los DSL, centrados en el ámbito de creación de software para audio, pero extrapolables a cualquier otro ámbito, reside en dos puntos clave:

1. Todo elemento, tanto de análisis como de diseño, debe tener una relación con algún otro.

2. El análisis de un proyecto ya no puede ser un conjunto de textos que se relacionan semánticamente con los artefactos de diseño. Para que un modelo de razonamiento automático pueda trabajar de forma eficiente, el análisis debe estar estrictamente configurado, y sus relaciones deben ser explícitas.

La herramienta desarrollada, MDAA (Model Driven Development Audio), imprescindible para que las gramáticas DSL desarrolladas resulten usables en desarrollos reales, se encuentra también ampliamente documentada en el repositorio del proyecto (véase Manual MDAA[11]). La idea clave de esta herramienta se puede resumir en los siguientes tres puntos:

1. Debe servir para que un analista, especializado en el dominio de aplicación (el desarrollo de sintes de audio en este caso), trabajando sobre un alto nivel de abstracción, pueda ser capaz de entender e implementar los DSL CIM desarrollados.

2. Debe servir para que un diseñador, especializado en el dominio de aplicación (sintetizadores de audio), trabajando ya en un nivel más bajo de abstracción, pueda ser capaz de entender e implementar los DSL PIM desarrollados.

3. Debe proporcionar una forma sencilla de poder generar los artefactos necesarios para que un modelo de razonamiento automático pueda generar el código fuente requerido.

[10] Manual DSL en GitHub

[11] Manual MDAA en GitHub

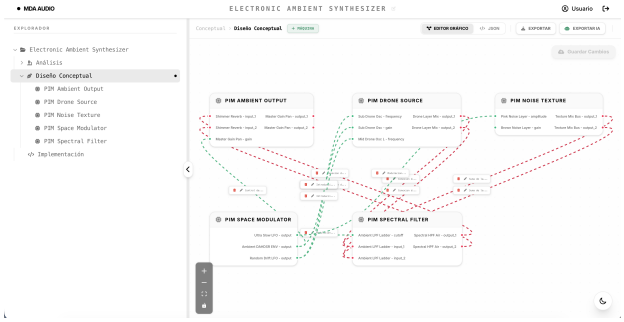

Figura 2: Relaciones entre máquinas, en el nivel de diseño conceptual (PIM), mediante la herramienta MDAA.

V. Descripción detallada de las herramientas generadas

En el repositorio del proyecto (véase Repositorio en GitHub[12]), se incluye un fichero `README.md` que resume todo el trabajo realizado respecto de las herramientas implementadas y su uso. Esta guía también sirve como base para navegar por los distintos apartados, scripts ejecutables y manuales de las herramientas implementadas. Por otro lado, el código fuente tanto de los DSL como de la herramienta MDAA se encuentra exhaustivamente documentado, empleando el estándar Javadoc para el código desarrollado en Java (tecnología usada para el *backend*) y JSDoc para las secciones implementadas en VUE/TypeScript (tecnología usada para el *frontend*).

Para la ejecución local del proyecto MDAA, necesitaremos tener instalado docker compose. Como se podrá ver en el *README.md* antes mencionado, tendremos una serie de scripts que facilitarán tanto la ejecución de la herramienta como la comprobación de los DSLs desarrollados, así como utilizades como el incluir los casos de uso en el entorno local instalado. Para ejecutarlos, se necesita una terminal linux-like (como puede ser la terminal de Linux, WSL de Windows, o la terminal de MacOS). Para la ejecución, compilación y validación de los DSL, como del script `validar-test.sh` (también comentado en el *README.md* antes mencionado), se necesita tener instalado nodejs. Por último, para el script de instalación de lso casos de prueba en la aplicación local, necesitaremos tener instalado tanto python 3.x como el conector mysql para python (`pip3 install mysql-connector-python`).

V.A. *Casos de uso aplicados para las pruebas de demostración.*

En la carpeta de casos de prueba del repositorio del proyecto (véase Casos de Prueba[13]), se encuentran documentados todos los casos de prueba realizados para la evaluación de la solución aportada. También se incluye un fichero *README.md*, comentando sus usos y formas de validación.

[12] Repositorio en GitHub

[13] Casos de Prueba en GitHub

VI. Evaluación y comparación de la solución

A continuación, se aplicarán llos criterios de evaluación, previamente definidos, a los cuatro casos de prueba ubicados en el repositorio (cuyos detalles se pueden consultar en el archivo *README.md*[14] correspondiente).

Las pruebas de generación se realizarán utilizando el modelo de IA Gemini PRO. Los códigos a evaluar corresponden al `.zip` generado para el lenguaje Pure Data a través de la aplicación MDAA. Asimismo, estos cuatro casos de prueba se encuentran, ya cargados bajo el usuario proporcionado, disponibles en el entorno desplegado en Azure[15] ya mencionado previamente (Usuario: `test@uned.es`, Contraseña: `11111111`).

A continuación, se presenta una tabla resumen de los resultados obtenidos:

Caso de Prueba	**Replicabilidad**	**Grado de Calidad**	**Número de Intentos**
Electronic Ambient	Muy buena	Correcto	2
Drum Machine	Excelente	Correcto	1
Arpeggio Bass	Muy buena	Correcto	2
80s Deep Bass	Excelente	Correcto	3

Cuadro 1: Evaluación de casos de prueba con Gemini PRO para Pure Data

De los resultados obtenidos, aunque preliminares y con un sesgo relativamente subjetivo, cabría destacar el apartado de replicabilidad, a través del cual se puede entrever que la solución aportada por MDAA es capaz de generar información suficiente para que los modelos entiendan lo que se persigue. Por otro lado, viendo los resultados obtenidos para el número de intentos, queda claro que, como ya se adelantaba, Pure Data no es un lenguaje óptimo para ser generado por modelos de razonamiento generalistas. Este problema es bien conocido en el ámbito del desarrollo con Pure Data, y proyectos como PatchGen[16] intentan solventarlo, con resultados muy interesantes, a través de entrenar (*fine-tuning*) modelos de poco tamaño, que posibilitan su ejecución en entornos locales, con ejemplos de *patches* desarrollados por la comunidad Pure Data.

[14] README de Casos de Prueba en GitHub

[15] `http://mdaa-app.spaincentral.azurecontainer.io/`

[16] `https://github.com/AnxiousAnt/PatchGen`

VII. Conclusiones y Trabajos futuros

La herramienta MDAA permite, de forma relativamente sencilla, implementar un flujo de trabajo MDA en el ámbito de la programación de sintetizadores de audio, proporcionando resultados muy interesantes al permitir la generación de código de forma automatizada y brindar trazabilidad entre sus distintos niveles de abstracción. Sin embargo, la herramienta está incompleta; solo brinda la posibilidad de que, tras la generación automática de una serie de ficheros estáticos, un modelo de razonamiento externo a ella sea capaz de generar el código fuente deseado. La evolución natural de esta herramienta debería ir enfocada a potenciar su autonomía, sin depender de modelos externos para la obtención de sus resultados finales e incluyendo en ella modelos locales entrenados para tal fin, que puedan encargarse también de la transformación automática entre los modelos DSL. MDAA parece brinda una base conceptual sólida para ello, y su evolución podría ser un interesante proyecto de fin de máster. Hay que tener en cuenta que MDAA es una herramienta en versión *Alfa* todavía, por lo que quedaría mucho trabajo por realizar en ella para que pudiera ser considerada una herramienta robusta y mantenible. Las gramáticas DSL para el ámbito en testeo sí se pueden considerar maduras, en el sentido de que cubren la mayor parte de lo que un software de rango pequeño o mediano de este tipo suele requerir. Aun así, también se considera que pueden ser ampliamente mejorables, ya no en su estructura base, sino en las capacidades que ofrecen.

Recalcar también que, aunque el artículo se centra en un dominio específico, desarrollo de sintetizadores de audio software, la estructura de los DSL desarrollados, desde un punto de vista de alto nivel, enfocando esta mirada en su idea de definición de *entes* (máquinas para el dominio ejemplo) en un DSL especifico, y sus interconexiones en otro distinto pero fuertemente relacionado, tanto para los niveles de abstracción de análisis como de diseño, parece un interesante punto de partida para su posible generalización, pudiendo partir de ese esquema inicial en el diseño de herramientas similares en otros ámbitos.

En definitiva, se considera este trabajo como un proyecto inicial, a completar, que podría abrir una línea de investigación interesante para futuros trabajos.